TECHNOLOGICAL UNIVERSITY DUBLIN

MASTERS THESIS

Enhancing Static Malware Analysis with Large Language Models and Retrieval-Augmented Generation

Author:
Andre M M FARIA

Supervisor:
Dr Robert G SMITH

A thesis submitted in fulfillment of the requirements
for the degree of M.Sc
in Applied Cybersecurity

in the

School of Informatics and Cyber Security

May 2025

Declaration of Authorship

I, Andre M M FARIA, declare that this thesis titled, "Enhancing Static Malware Analysis with Large Language Models and Retrieval-Augmented Generation" and the work presented in it are my own. I confirm that:

- This work was done wholly or mainly while in candidature for a research degree at this Institute of Technology Blanchardstown.
- Where any part of this thesis has previously been submitted for a degree or any other qualification at this University or any other institution, this has been clearly stated.
- Where I have consulted the published work of others, this is always clearly attributed.
- Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this thesis is entirely my own work.
- I have acknowledged all main sources of help.
- Where the thesis is based on work done by myself jointly with others, I have made clear exactly what was done by others and what I have contributed myself.

Signed:

Date: August 24, 2025

"The true masters are the friends we make along the way."

Anonymous

TECHNOLOGICAL UNIVERSITY DUBLIN

Abstract

School of Informatics and Cyber Security

M.Sc

Enhancing Static Malware Analysis with Large Language Models and Retrieval-Augmented Generation

by Andre M M FARIA

Malware's quick growth presents serious cybersecurity concerns, necessitating ongoing innovation in methods for detection, analysis, and mitigation. When it comes to handling complex and adaptable threats, traditional malware analysis techniques, which mostly rely on manual labour and rule-based systems, are progressively less effective. Large language models (LLMs) are a revolutionary way to automate malware analysis, even though Artificial Intelligence (AI) approaches have been effectively incorporated into cybersecurity. Current AI solutions, including machine learning classifiers and clustering algorithms, concentrate on aspects of malware but frequently lack the overall skills necessary to manage intricate datasets or fully understand virus behaviors.

This study suggests using LLMs to improve and automate static malware analysis. It seeks to automate crucial components of malware reverse engineering, such as code analysis, possible behavior interpretation and anomaly detection, by leveraging LLMs' capacity to handle and synthesize massive, heterogeneous information. In addition to addressing the drawbacks of manual and conventional AI techniques, this strategy adds scalability and adaptability to quickly changing malware threats.

In order to optimize the efficacy of malware analysis in cybersecurity, this study proposes a methodology that combines domain-specific datasets, such as malware sample databases, with prompt engineering techniques to evaluate samples and identify parameters such as malware classification categories and behavioral patterns on decompiled code. This approach is expected to provide a more comprehensive and accurate analysis of malware samples, as well as to improve the overall efficiency of malware analysis in cybersecurity.

Keywords: Malware Analysis, Large Language Models, Static Analysis, Cybersecurity, Artificial Intelligence, Generative Artificial Intelligence, Machine Learning, Reverse Engineering, Anomaly Detection, Code Analysis.

Acknowledgements

Thanks to my parents for supporting me through everything over the years…

Thanks to my supervisor, Dr Robert G Smith, for his guidance and support. :D

Contents

List of Figures

List of Tables

Chapter 1

Introduction

Ever since the creation of the first computer by Charles Babbage in the 19th century, computers have been evolving at a rapid pace performing more and more tasks as the time passes. With this evolution, malicious actors began to notice manners in which to subvert established systems. These individuals, by force of belief or personal gain, have caused losses in the trillions to organisations and individuals across the globe.

As a response to these threats, these organisations and individuals began to develop techniques and tools to counter malicious actors. Some of these techniques revolve around the analysis of what software adversaries develop. The analysis of such malicious software is important in order to study and understand how they operate and the vulnerabilities they exploit to weaken, or outright topple, established security systems.

This research presents a the development of a methodology to analyse malicious software using artificial intelligence (AI) tools to enhance analysis quality and speed. This tools will be discusses toughly on 3.

This thesis presents a research about, and creation of, a method where RAG powered LLMs are introduced onto to the malware static analysis workflow in order to increase efficiency on the analysis.

1.1 Background and Motivation

1.1.1 Static Malware Analysis in Cybersecurity

These software artifacts created by malicious actors are called Malware. Static malware analysis focus specifically on techniques that decompose, or decompile, target Malware down to its most basic bytecode elements. These are then scrutinized to uncover malicious intent. This method of inspecting a program's structure, instructions, and information without actually running it, is one of the most popular and dependable tools in the cybersecurity toolbox. It is essential in settings where safety, speed, and scalability are critical since it eliminates the possibility of running potentially dangerous code. It is the keystone in early malware identification and categorization, due to its automation capabilities (i.e. antivirus engines or intrusion detection systems).

However, as threat actors evolve their tactics and adopt increasingly sophisticated evasion techniques, static analysis has begun to show its limitations. Many contemporary malware variants leverage obfuscation, packing, encryption, and code polymorphism to mask their functionality and frustrate traditional static inspection. Moreover, extracting actionable intelligence from static features often requires expert-level knowledge in assembly language, file formats, and API behaviour—creating a steep barrier for automation. These challenges have shown

the need for intelligent, context-aware tools capable of shortening the gap between low-level code patterns and high-level semantic understanding.

1.1.2 Emergence of LLMs in Code and Text Understanding

Over the past few years, Large Language Models (LLM) have rapidly transitioned from experimental research tools to foundational components across a wide range of natural language processing and code analysis applications. Built on transformer-based architectures and trained on vast corpora comprising not only natural language text but also programming languages and technical documentation, these models have demonstrated an impressive capacity to generate, summarize, translate, and reason about code and human language alike. Their success in handling structured and semi-structured data has opened new avenues in fields such as automated code completion, bug detection, documentation generation, and even code synthesis. With models like GPT, CodeBERT, and StarCoder pushing the limits of scale and capability, LLMs are starting to play a major part in determining how we interact with intricate codebases and interpret syntactic patterns that once required domain-specific knowledge.

This increased capacity in both textual and programmatic reasoning has generated interest in bringing LLMs to cybersecurity, where duties such as code interpretation, vulnerability identification, and malware classification have historically been left for highly trained analysts. Unlike rule-based systems, which rely on predetermined signatures or static heuristics, LLMs may generalise from a variety of examples and adapt to novel or obfuscated code patterns, which is very valuable in malware research. Furthermore, their capacity for natural language explanation enables a level of interpretability that aligns well with security workflows that demand transparency and traceability. While LLMs have shown early promise in tasks such as vulnerability detection and reverse engineering assistance, their full potential in enhancing static malware analysis, especially through integration with contextual retrieval systems, remains a largely untapped area of research, presenting an opportunity to reimagine traditional security paradigms with the help of advanced, language-aware intelligence.

1.1.3 Motivation

The reasoning behind the focus of this research is that static analysis remains to be a critical component of malware detection and neutralization. Security analysts are often required to analyze and process huge amounts of obfuscated data. By integrating RAG powered LLMs to this process we can make the analysis easier by tapping onto contextual knowledge from the RAG and the analysis power of LLMs.

1.1.4 Key Concepts

Here are defined all of the concepts and/or tools that are mentioned.

1.1.4.1 Large Language Models

Large Language Models (LLMs) are a type of multi-parameter Machine Learning (ML) algorithm designed for natural language processing. These models that are trained on a large volume of text using a multitude of learning techniques such as supervised and self-supervised learning.

Generative pre-trained transformers (GPTs) are the biggest and most powerful LLMs. Current models can be adapted to certain tasks (via fine-tuning) or directed by prompt engineering [[**brown2020languagemodelsfewshotlearners**]]. These models learn to predict the syntax, semantics and ontologies found in human language, but they also pick up biases and errors from the training data [[**10.1162/daed_a_01905**]]. Popular examples of LLMs include:

- GPT models (like GPT-4 or ChatGPT)
- Google's PaLM and Gemini
- Meta Llama series models

1.1.4.2 Prompt Engineering

Interactions with LLMs are made through *prompts*. Those are natural language text segments that describe the task the AI should perform [[**radford2019language**]]. They can be queries, commands or statements including context, instructions and/or relevant information.

Prompt engineering is the action of manipulating these text segments in order to optimize the wanted output [[**wahle-etal-2024-paraphrase**]]. For example, providing a prompt such as "What is an apple?" we have this output:

> An apple is a round, edible fruit produced by the apple tree (Malus domestica). It typically has smooth skin that can be red, green, or yellow, and a sweet to tart taste depending on the variety. Apples have a firm white flesh and contain a core with small black or brown seeds.
> They're commonly eaten raw, but also used in cooking and baking-like in apple pies, sauces, and juices. Apples are rich in fiber, vitamin C, and antioxidants, making them a popular choice for a healthy snack. The saying "an apple a day keeps the doctor away" hints at their reputation for promoting good health.
> Want to know about their origin, symbolism, or maybe how they're grown?

This output is acceptable, however it can be modified with some prompt engineering, such as an ask to change the output formatting. Then, providing the phrase "Provide only bullet points." in addition ot the original we have:

> * Edible fruit from the apple tree (Malus domestica)
> * Typically round with red, green, or yellow skin
> * White, crisp flesh with a sweet to tart flavor
> * Contains a core with small seeds
> * Commonly eaten raw or used in cooking and baking
> * Rich in fiber, vitamin C, and antioxidants
> * Grown in temperate climates worldwide
> * Symbol of health, knowledge, and temptation in various cultures

1.1.4.3 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) [[13]] is a technique that allows for LLMs to query information from a specific type of datastore and use it as context for the response it provides. This allows for adaptive learning of the models without the

need of retraining (fine-tuning). It improves the quality of LLM output as the model does not rely solely on pre-trained knowledge but also on relevant information from the datastore.

RAG works by having the LLM to query the datastore using vector search, embeddings or keyword matching (depending on the datastore type), feeding this data onto the LLM input and then continuing with the normal LLM process for output generation.

1.1.4.4 Malware

Short for *Malicious Software*, Malware is any program or code that is created for malicious purposes like exploitation of computer systems networks or users. Often created with monetary gain in mind, these programs activities include stealing sensitive data, damage systems, disrupt operations or gain unauthorized access to systems or environments. Common types of malware include:

- Viruses: Attach themselves to legitimate software, spreading when the software is run.
- Worms: Autonomous malware capable of multiplying across networks without needing human intervention. Exploit network weaknesses to infiltrate other systems without permission.
- Trojans: Also known as Trojan Horses, these are disguised as genuine software to trick users into running them.
- Ransomware: Encrypts or locks files, demanding payment (ransom) for restoration.
- Spyware: Stealthily gathers sensitive data and user activities without the user's consent.
- Adware: Delivers unwanted advertisements, potentially slowing down or compromising systems.

1.1.4.5 Malware Analysis

As mentioned before, the software created by malicious actors needs to be evaluated and analyzed to understand how it works, what exploits it takes advantage of and how these exploits can be patched. As a basis, there are two types of analysis of software that can be made to understand any kind of software. These are as follows:

- **Static Analysis**: Focuses on looking at the sequences of individual instructions on the software bytecode to identify the its characteristics, such as identification of the parameters in which the analyzed software is built upon (e.g. Operating system it executes on, processor architecture, etc), what execution paths it employs, what system calls it performs, etc. This analysis is done without running the code and it requires a lot of time and effort from the analyst.
- **Dynamic Analysis**: Entails running the potential malicious software to gather runtime information such as network calls and system call execution data which are not visible through static analysis. This analysis is done by creating a special (sandbox) environment (i.e docker container and/or a virtual machine) and running the software.

1.1.4.6 Static Analysis vs Dynamic Analysis

Both static and dynamic analysis have their own benefits and are important on their own right for understanding how malware is designed and developed. While static analysis takes advantage of its speed and safety where files do not need to be executed to be analyzed, dynamic analysis enables a more extensive analysis which might not be visible with static analysis alone.

However, both have their limitations such as the increased computational cost and requirement of a secure environment of dynamic analysis versus the vulnerability of static analysis to obfuscation and encryption techniques.

1.2 Problem Statement

TODO...

1.3 Research Aim and Objectives

1.3.1 Aim

Research the enhancement of static malware analysis using RAG-enhanced LLMs

1.3.2 Objectives

- Develop a method for integrating LLMs with RAG to enhance static malware analysis.
- Evaluate the effectiveness of RAG-enhanced LLMs in identifying, explaining, and comparing malware threats using static features (e.g., file structure, bytecode, API calls) against traditional static malware analysis techniques in terms of accuracy, efficiency, and interpretability, while identifying challenges and potential risks (e.g., adversarial manipulation and hallucination).

1.4 Research Questions and Hypothesis

1.4.1 Questions

These are the research questions that will guide the development of this thesis.

- How LLMs can be leveraged to enhance static malware analysis?
- How does a RAG-enhanced LLM compare to traditional static analysis techniques in terms of accuracy, efficiency, and interpretability?
- What role does RAG play in improving LLM-driven malware classification, particularly in terms of contextual relevance and justifiability of results?

1.4.2 Hypothesis

- Hypothesis 1: LLMs can enhance static malware analysis by accurately interpreting and classifying malware samples based on static features such as bytecode, file structure, and API calls, offering greater adaptability and insight than rule-based static tools.

- Hypothesis 2: A RAG-enhanced LLM will outperform traditional static analysis techniques in terms of interpretability and contextual understanding, while maintaining comparable levels of accuracy and efficiency in malware classification.
- Hypothesis 3: RAG improves LLM-driven malware classification by providing relevant contextual information during inference, leading to more justifiable and semantically grounded explanations of classification results.

1.5 Methodology Overview

TODO...

1.6 Contributions of the Research

TODO...

1.7 Thesis Structure

TODO...

Chapter 2

Literature Review

This chapter presents the current state of the art on static malware analysis, with a focus on the integration of Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG) to enhance automation, semantic understanding, and explainability in malware detection workflows. Static analysis plays a crucial role in cybersecurity, enabling the examination of malicious code without execution. Its safety, repeatability, and efficiency make it essential for malware triage and reverse engineering tasks [6]. However, challenges such as code obfuscation, semantic ambiguity, and limited contextual awareness continue to limit its effectiveness [9, 5].

Traditional approaches, such as signature-based and heuristic techniques, have been widely deployed in antivirus engines. These methods depend on predefined rules and known byte patterns, which are easy to circumvent with packing or polymorphic transformations [18]. In response to these limitations, Machine Learning (ML) and deep learning models have been introduced to detect and classify malware based on statistical features and behavior signatures extracted from static data [5, 9]. Despite notable improvements in detection rates, many of these models require extensive feature engineering, struggle with generalization, and lack transparency regarding decision-making processes.

Recently, the emergence of large-scale language models such as GPT-4 has opened new avenues for augmenting static malware analysis. Fujii and Yamagishi [8] demonstrated that LLMs can explain decompiled functions in plain language and provide analysts with faster contextual insights, particularly when guided with structured prompts or roles. These findings support the use of Model-Centric Prompting (MCP) as a methodology for optimizing prompt engineering in code-focused tasks [20]. However, their work also exposed limitations in accuracy and hallucination when insufficient context is provided.

RAG offers a promising solution to these challenges by dynamically incorporating external information—such as threat reports or similar malware case studies—into the prompt context [10]. While RAG has shown success in NLP applications like question answering and summarization, its potential in malware analysis remains underexplored, representing a key research opportunity identified in this study [13].

This literature review synthesizes current efforts across five key themes:

- Fundamentals and mechanisms of LLMs and RAG.
- Characteristics of malware and the importance of static analysis.
- Applications and limitations of traditional and ML-based static malware analysis.
- Existing and emerging roles of LLMs in cybersecurity.

- Opportunities for integrating RAG and LLMs to enhance analysis accuracy and explainability.

2.1 Static malware Analysis

Static malware analysis refers to the examination of malicious software without executing it, typically by inspecting its binary code, structure, metadata, and associated artifacts. It remains a foundational practice in cybersecurity due to its safety, speed, and reproducibility [6, 9, 5]. Analysts use this method to infer program functionality, extract Indicators Of Compromise (IOCs), and generate initial threat intelligence without the risk of executing potentially harmful code.

Basic static analysis methods include file header inspection (e.g., PE header metadata), string extraction, and checksum analysis. Advanced techniques rely on disassembly, Decompilation, and control flow graph analysis to reconstruct higher-level logic [6, 22]. These techniques provide insights into what the malware does and how it operates, but suffer from challenges such as obfuscation, code packing, and the absence of runtime context.

Despite their limitations, static methods offer advantages over dynamic analysis, especially for rapid triage and offline forensic evaluation. They are immune to anti-virtualization and anti-debugging tactics often used by sophisticated malware. However, modern threats often employ techniques like runtime linking, encrypted payloads, and evasion frameworks, limiting the reach of purely static tools [6, 22].

These limitations have created interest in augmenting static analysis with intelligent reasoning systems, especially LLMs, which are explored in the following sections.

2.1.1 Signature-Based and Heuristic Methods

Signature-based analysis matches specific byte patterns, string literals, or opcode sequences in a binary against known malware fingerprints. This method is fast and widely used by analysis tools such as *ClamAV*, *YARA*, and *VirusTotal* [6]. However, even slight changes to the malware's structure—such as packing or encryption—can render signatures ineffective.

Heuristic analysis improves on this by flagging suspicious patterns based on known behaviors or structural anomalies [18]. For example, unusual import tables, abnormal section sizes, or the use of specific API calls (e.g., `VirtualAlloc`, `CreateRemoteThread`) are often considered red flags. Yet these rule-based systems are limited by their brittleness and potential for false positives [9, 5].

2.1.2 Machine Learning-Based Approaches

To overcome the rigidity of traditional methods, static malware analysis has increasingly incorporated ML [5]. These approaches use feature representations—such as n-grams, opcode frequencies, or entropy values—to train classifiers capable of detecting both known and unknown malware samples. Datasets like EMBER, Malimg, and VirusShare facilitate these developments by providing labeled corpora for training and evaluation [9, 2].

However, challenges remain. Feature engineering often requires expert domain knowledge, and deep learning models can be opaque or hard to interpret. More importantly, many ML-based detectors struggle with obfuscated or packed malware where feature extraction fails to surface meaningful signals [5].

2.1.3 Practical Considerations and Analyst Workflows

According to field studies such as Wong *et al.* [22], malware analysts adopt a tiered approach to static analysis, starting with basic tools like hash extractors and moving toward disassembly and Decompilation as needed. Tools like `Ghidra`, `IDA Pro`, and `Radare2` are central to this process, offering powerful Decompilation and navigation features, albeit requiring significant expertise.

Static analysis of PE (Portable Executable) files—commonly used by Windows binaries—reveals critical metadata such as entry points, import address tables, and linked libraries. This data provides early insights into malware behavior, even before Decompilation [6].

Fedák and Štulrajter [6] provide detailed insight into the practical toolkit of an analyst, listing tools such as:

- **Strings**, **HexDive**, and **BinText** for readable text extraction.
- **PEiD** and **Exeinfo PE** for identifying compression techniques and signatures.
- **CFF Explorer** and **Dependency Walker** for PE header inspection and DLL analysis.

2.1.4 Obfuscation, Compression, and Runtime Challenges

Modern malware often leverages runtime linking and obfuscation to bypass static detection. Code is compressed or encrypted, with only loader stubs visible to analysts. These stubs typically use runtime API resolution (e.g., via `LoadLibrary`, `GetProcAddress`) to dynamically fetch and execute malicious payloads [6].

The prevalence of obfuscation significantly reduces the efficacy of basic static tools. For example, string extraction becomes nearly useless if meaningful strings are encrypted. In such cases, unpacking and de-obfuscation—either manual or using tools like Exeinfo PE—are essential to enable deeper analysis [22].

2.1.5 Summary of Limitations and Opportunities

While static malware analysis remains a cornerstone of threat detection, its limitations are well-documented [22, 6]:

- Lack of semantic understanding (e.g., what the malware does or why it behaves that way).
- High dependency on manual expertise for effective use of tools.
- Vulnerability to evasion through obfuscation, packing, and runtime behavior.

These limitations have motivated the exploration of Artificial Intelligence (AI) driven approaches that promise greater automation, semantic reasoning, and contextual interpretation.

2.2 The Role of Language Models in Cybersecurity

LLMs have rapidly become key enablers in cybersecurity, offering powerful automation for tasks such as malware detection, vulnerability analysis, code auditing, incident summarization, and threat intelligence extraction. Pre-trained on diverse code and text, LLMs like `GPT`, `BERT`,

`CodeBERT` [**devlin2019bertpretrainingdeepbidirectional**] and `Codex` show impressive generalization and reasoning over both structured and unstructured data [17, 20, 7, 16].These models are built upon the Transformer architecture, introduced by Vaswani et al. [21], which uses self-attention mechanisms to model global dependencies efficiently and supports highly parallelizable training.

In the domain of malware analysis, LLMs are particularly useful for interpreting decompiled or disassembled functions and generating natural-language summaries. Fujii and Yamagishi [8] demonstrated that GPT-4 can accurately explain functions when prompted appropriately—especially using role-based prompts such as "You are a malware analyst…." This highlights the importance of Model-Centric Prompting (MCP), where prompts are tailored to align with model reasoning behavior rather than merely input format [20].

Recent surveys such as Nourmohammadzadeh *et al.* [17] and Ferrag et al. [7] show growing interest in applying LLMs to cybersecurity, but also emphasize that these models require adaptation to operate on binary code, disassembled structures, and malicious behavior patterns. The research by Metta et al. [16] provides a comprehensive risk-centric perspective, highlighting both defensive and offensive uses of LLMs—from malware analysis and reverse engineering to code generation for adversarial purposes.

Key benefits of LLMs in cybersecurity include:

- **Automated explanation and triage:** translating complex code into natural language.
- **IOC and pattern extraction:** identifying suspicious API calls, encoded payloads, or unusual imports.
- **Augmenting human analysts:** supporting SOC workflows through summarization and prioritization.

However, several limitations persist:

- **Hallucination and trust:** LLMs may confidently produce incorrect or fabricated information, especially without sufficient context.
- **Prompt sensitivity:** output quality varies based on structure and phrasing [20].
- **Black-box reasoning:** difficulty auditing or interpreting why a model made a specific decision [16].
- **Security concerns:** models may be vulnerable to prompt injection, adversarial inputs, or training data poisoning.
- **Opacity:** The internal reasoning of LLMs remains a "black box," limiting trust and auditability [17].

Ferrag *et al.* (2024) further explore the vulnerabilities and risks associated with deploying LLMs in cybersecurity workflows, including prompt injection, data poisoning, and adversarial manipulation—even as they highlight methods for adaptation like fine-tuning, retrieval augmentation, and reinforcement learning [7].

Despite these challenges, LLMs contribute meaningfully to cybersecurity by reducing cognitive load and accelerating analysis [7]. When combined with techniques like RAG, their outputs can be better grounded, more reliable, and transparently justified.

2.3 RAG in NLP

RAG is a technique that enhances the performance and factuality of LLMs by supplementing them with external, retrievable knowledge sources. Instead of relying solely on pre-trained parameters, RAG models incorporate a retrieval module that fetches relevant context—such as documents, reports, or structured data—which is then used to condition the generative response. This paradigm is particularly beneficial in knowledge-intensive tasks such as question answering, summarization, and factual reasoning [13, 10].

Lewis *et al.* [13] introduced the original RAG framework by combining a dense retriever with a seq2seq LLM, demonstrating improvements in open-domain QA by grounding outputs in retrieved passages. Subsequent research has refined the retrieval mechanisms (e.g., dense vs. sparse, hybrid models), document chunking strategies, and integration techniques, leading to significant gains in factual correctness and reduced hallucinations across tasks.

Gao *et al.* [10] provide a comprehensive survey of RAG systems and identify key benefits:

- **Factual accuracy:** Retrieved documents act as grounding references, reducing the likelihood of hallucinations.
- **Scalability:** Retrieval modules can be updated independently, avoiding the need to retrain the base model.
- **Context expansion:** RAG enables models to dynamically adapt to task-specific or domain-specific corpora.

These properties make RAG a compelling approach in domains where correctness, recency, and traceability of information are critical—such as medicine, law, and increasingly, cybersecurity.

In the context of malware analysis and static code reasoning, RAG holds significant potential: analysts often consult documentation, malware reports, or IOC databases during investigations. A RAG-augmented LLM could mimic this behavior by retrieving semantically relevant samples, threat profiles, or code analogues to provide better-informed and more explainable outputs.

The security of RAG-based systems is an emerging concern. Zou et al. [24] introduced the *PoisonedRAG* threat model, showing that adversaries can corrupt the retrieval index to manipulate model outputs via malicious documents. Their findings underscore the need for trusted, sanitized retrieval pipelines and integrity checks when applying RAG in high-stakes domains like cybersecurity.

Despite these risks, RAG remains a promising solution to many of the context limitations identified in static malware analysis. It enables LLMs to ground their outputs in verifiable sources, improve justifiability, and reduce hallucination—all of which align with the goals of this thesis. The next section explores the conceptual integration of RAG and LLMs for static malware analysis and highlights emerging research directions at this intersection.

2.4 Combining rag and LLMs for Static malware Analysis

While LLMs have demonstrated utility in malware analysis tasks such as code explanation, IOC extraction, and classification, they are often constrained by limited context windows and the absence of external knowledge. RAG offers a

natural extension to overcome these limitations by dynamically supplying relevant, grounded information during inference.

Static malware analysis workflows frequently rely on external resources—such as malware repositories, IOC databases, Decompilation guides, threat reports, and *YARA* rule libraries—to contextualize findings. A RAG-enhanced LLM can emulate this expert behavior by retrieving semantically similar samples, annotated analyses, or supporting documentation as prompt context [13, 10]. This contextual grounding enables the model to:

- Improve detection accuracy for obfuscated or novel malware by comparing with similar known variants.
- Enhance interpretability through grounded justifications drawn from previous samples or reports.
- Reduce hallucinations by anchoring generation in retrievable, verifiable content.

Although few studies have explicitly combined RAG with static malware analysis pipelines, early evidence in adjacent fields supports this integration. In source code understanding and vulnerability detection, retrieval-enhanced models improve performance by supplying relevant snippets or past issue reports to the model at inference time. This inspires analogous use in static malware contexts, where samples with similar bytecode structures or behavioral markers can be retrieved and used to interpret new or ambiguous binaries.

Moreover, Fujii and Yamagishi [8] highlighted that LLM hallucination increases when insufficient context is provided to the model, especially in malware-specific tasks. Integrating RAG directly addresses this issue by enriching the prompt with retrieved knowledge, which can include:

- Decompiled code from known malware families.
- Threat intelligence reports on similar IOC patterns.
- Descriptions of suspicious API usage from documentation.

Such retrievals not only make model outputs more reliable but also provide analysts with traceable reasoning paths. When paired with structured prompt design methodologies like Model-Centric Prompting (MCP), the combined RAG-LLM approach could significantly improve automation and accuracy in static analysis pipelines.

Security and reliability challenges remain. As noted by Zou *et al.* [24], poisoned documents or manipulated retrieval indices can lead to knowledge corruption and misleading outputs. To mitigate these risks, trusted retrieval indices, curated malware corpora, and retrieval integrity checks should be considered foundational elements of any cybersecurity-focused RAG pipeline.

The integration of RAG and LLMs into static malware analysis pipelines is an emerging and largely untapped area. Its benefits—contextual grounding, improved explainability, and reduced hallucination—are directly aligned with the needs of modern malware analysts.

2.5 Gaps in Current Literature

The reviewed literature illustrates steady progress in static malware analysis and the adoption of AI in cybersecurity. However, several significant gaps persist—both technical and methodological—that motivate this research.

2.5.1 Lack of Integration Between rag and malware Analysis

While RAG has demonstrated success in natural language tasks such as question answering and summarization [13, 10], its application in malware analysis—particularly static workflows—remains largely unexplored. No current work bridges LLM-driven reasoning with contextual retrieval of prior malware samples, reports, or IOC patterns, despite its strong theoretical fit.

2.5.2 LLM Use is Largely Exploratory and Unbenchmarked

LLM-based research in static analysis is still in its infancy. Studies like Fujii and Yamagishi [8] provide qualitative results demonstrating feasibility, but there is a lack of standardized benchmarks, evaluation frameworks, or comparative studies with traditional tools such as YARA or ML classifiers. The GenAI in Cybersecurity report [16] also notes the scarcity of domain-specific evaluation methods that account for explainability and analyst trust.

2.5.3 Explainability, Auditability, and Analyst Trust Are Underserved

Interpretability is essential for analysts making high-stakes security decisions [22]. However, most AI-driven malware analysis models—including LLMs—fail to provide transparent reasoning or traceable justifications. This issue is compounded by hallucinations and black-box behaviors, limiting practical adoption [16, 20].

2.5.4 Underutilization of Prompt Engineering and Interaction Protocols

Effective use of LLMs requires structured prompt design. Yet, most cybersecurity applications ignore prompt engineering methodologies like Model-Centric Prompting (MCP), which are shown to improve output reliability and task alignment [20]. A lack of formalized interaction strategies hinders reproducibility and evaluation.

2.5.5 Emerging Security Risks in AI-Augmented Pipelines

As outlined by Zou *et al.* [24] and echoed in the generative AI security report [16], RAG-based and LLM-powered systems face risks such as data poisoning, prompt injection, and misinformation amplification. None of the surveyed malware analysis frameworks account for these threats, indicating a serious oversight in current literature.

By addressing these gaps, this research seeks to develop a reproducible framework that combines RAG, LLMs, and structured prompts to enhance static malware analysis.

2.6 Summary

This chapter has surveyed the state of the art in static malware analysis, the emerging role of LLMs in cybersecurity, and the promising capabilities of RAG in improving contextual reasoning and output reliability.

We began by reviewing traditional and machine learning-based static analysis techniques, highlighting their advantages in safety and scalability but also their limitations—particularly in handling obfuscation, semantic interpretation, and novel threats [6, 9, 5]. Next, we explored how LLMs offer new possibilities for automating and interpreting malware analysis, while also facing challenges related to hallucination, prompt sensitivity, and explainability [8, 17].

RAG was introduced as a powerful strategy to address LLM limitations by supplying external context at inference time. Its benefits include improved factual grounding, reduced hallucination, and adaptability to evolving threat landscapes [13, 10]. However, its use in malware analysis remains nascent, and security concerns — such as retrieval poisoning — must be considered carefully in high-assurance settings [24].

Combining RAG with LLMs in the context of static malware analysis represents an underexplored research opportunity. The literature currently lacks comprehensive studies evaluating such integrations or establishing reproducible, explainable pipelines for cybersecurity tasks. This gap is further compounded by minimal usage of structured prompting methodologies like Model-Centric Prompting (MCP), which could improve the interpretability and robustness of AI-generated analyses [20].

The insights from this chapter establish the foundation for the proposed research. In the next chapter, a methodology is introduced to develop and evaluate a framework that combines RAG, LLMs, and structured prompts for static malware analysis — focusing on accuracy, explainability, and resilience.

Chapter 3

Methodology

This chapter explains the methodology used to build and evaluate a prototype system for static Malware analysis supported by LLMs and RAGs. The system runs two pipelines on the same set of samples. The first is a baseline classifier that combines VirusTotal tags with heuristics taken from Ghidra's static analysis, such as imports, functions, and strings. The second is an LLM-based pipeline that uses the same static features but also brings in external context through a retrieval component from a RAG enabled datastore. Running both pipelines in parallel makes it possible to compare accuracy, the relevance of supporting evidence, and the clarity of explanations.

The choice to use LLMs in this way builds on recent work by Fujii and Yamagishi [8], who showed that models like GPT-4o can interpret decompiled code and extracted metadata for classification tasks. Broader surveys [17, 7, 16] confirm the growing role of LLMs in security analysis, particularly where interpretability and contextual reasoning are important. The comparative setup also reflects the practitioner view described by Wong *et al.* [22], where simple, transparent heuristics remain central to Malware triage.

Using RAGs introduces new risks. As Zou *et al.* [24] note, retrieval pipelines can be poisoned or manipulated, leading to distorted outputs. To address this, the data for the corpus comes from a selection of trusted sources, including VX-Underground APT reports, MalwareBazaar submissions, and Malpedia documentation. The provenance of these sources reduces the chance of corrupted context and strengthens the reliability of the retrieval process.

3.1 Research Design

The research is experimental and exploratory, aimed at testing whether LLMs combined with RAGs can strengthen static Malware analysis. Instead of training a new model or fine-tuning existing ones, the study uses a **prototype-based dual-pipeline design** to compare two independent approaches under the same conditions:

- **Baseline Classifier:** A rule-based system that fuses VT tags with heuristics extracted from Ghidra. Features such as imports, functions, and strings are processed into rules that emulate the quick, interpretable triage used in analyst workflows [22].
- **LLM + RAG Classifier:** A pipeline built on Haystack (Python library), using PostgreSQL with pgvector to retrieve relevant context documents. Extracted features and retrieved context are combined into structured prompts and passed to an LLM (e.g., GPT-4o) for classification and explanation.

This design reflects earlier work by Fujii and Yamagishi [8], who showed that general-purpose LLMs can interpret decompiled code and static features to support classification. It also builds on Wong *et al.* [22], who describe the role of lightweight heuristics in everyday malware triage. The pipelines are evaluated strictly in **inference mode**, following Gao *et al.* [10], who highlight how RAGs can improve reasoning without retraining by injecting relevant external knowledge into prompts.

Because retrieval systems can be manipulated, the risks of hallucination or poisoning identified by Zou *et al.* [24] are addressed by strict control over data provenance: only curated, high-quality sources (VX-Underground, MalwareBazaar, and Malpedia) are ingested into the datastore. All outputs are logged for traceability and later comparison.

3.1.1 Experimental Workflow

Each sample is processed through both pipelines in parallel to produce outputs under identical input conditions. Those outputs follow a specific pattern outlined on the next chapter. This setup allows side-by-side benchmarking and highlights the contribution of LLM-based reasoning when compared to static heuristics alone.

The workflow for each sample is as follows:

1. **Static Feature Extraction:** Performed with Ghidra and processed into a structured form as can be seen in section 3.2.2.
2. **Pipeline 1 – Baseline Classifier:**
   - Extracted features are matched against heuristic rules (e.g., suspicious imports, entropy thresholds).
   - On demand, VT metadata are acquired (if available) and compared against the heuristic evaluation through a decision engine to standardized family labels when available.
   - Output includes classification, triggered heuristics, and a confidence score.
3. **Pipeline 2 – LLM + RAG Classifier:**
   - Extracted features are used to query the Haystack (Python library)+PostgreSQL+pgvector datastore.
   - Retrieved documents and features are embedded into a prompt template.
   - Prompt is passed to an LLM (e.g., GPT-4o) for inference.
   - Output includes classification, explanation, and reasoning trace.
4. **Logging and Storage:**
   - Both pipelines log their outputs, including:
     - Sample hash (SHA256)
     - Pipeline name (Baseline or LLM+RAG)
     - Classification result
     - Confidence score
     - Explanation or triggered rules
     - Timestamp and runtime metadata
   - All results are indexed by sample hash and timestamp for reproducibility and evaluation.

3.2 System Architecture

The system ingests and analyzes Malwares samples through a modular pipeline that allows reproducibility, parallel execution of different analysis methods, and detailed logging. As shown in Figure 3.1, the architecture is built around two pipelines that run side by side: a baseline classifier using static heuristics and VT metadata, and an LLM+RAG classifier that sums static features with retrieved context. Both receive the same input sample and output a classification with supporting metadata.

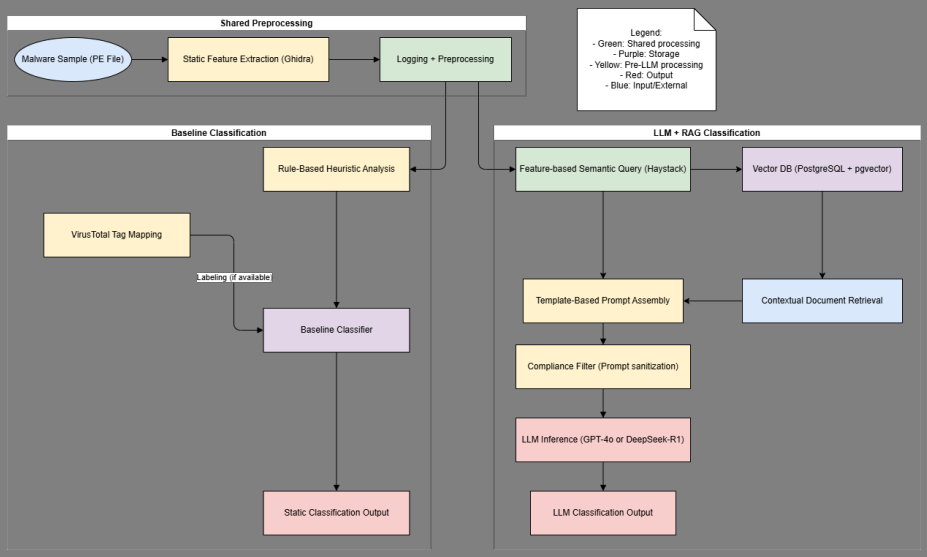

FIGURE 3.1: System Architecture Pipeline

3.2.1 Malware Sample Ingestion

The system accepts samples in Windows executables (Portable Executable or PE format) format, such as `.exe`, `.dll`, and `.sys` files. Each binary is indexed by its SHA256 hash and stored locally for analysis.

3.2.2 Static Feature Extraction with Ghidra

Static features are extracted through an automated Ghidra installation controlled with `pyghidra`. This setup supports batch decompilation and scripted analysis. Extracted features include:

- `Program information`: Summary metadata about the binary with fields like original file name, file format, binary architecture, file size, etc.
- `Functions summary`: Summary of the functions discovered in the binary.
- `Full code of decompiled functions`: Decompiled function representations for higher-level review.
- `Imported Functions/API`: Imported API/function names from external libraries/DLLs.
- `Strings`: Printable strings extracted from the binary.
- `Memory Sections`: Memory/PE sections with names, sizes, attributes (e.g., entropy).

- `Exported`: Symbols/functions the binary exports (if any).
- `Entry Points`: Entry point addresses or function names identified.

These features are logged to a structured datastore and passed to both pipelines. Using Ghidra ensures consistency across runs and provides visibility into sample internals.

3.2.3 Pipeline A – Baseline Classifier

The baseline pipeline reconciles two sources of evidence: static heuristics derived from Ghidra and tag information from VT. Extracted features are matched against heuristic rules (e.g., entropy thresholds, suspicious imports), producing an internal score and confidence. In parallel, VT engine results are aggregated into a consensus score with a confidence value based on coverage, recency, and naming consistency. A reconciliation layer then fuses these signals using a weighted approach that adjusts for conflicts, applies penalties for weak evidence, and can abstain when both sources disagree with high confidence. The output is a classification label, score, and explanation trace. This design provides a transparent reference point for comparison and reflects the reconciliation logic described in Section 3.4.1.

3.2.4 Pipeline B – LLM+RAG Classifier

The second pipeline integrates retrieval and model-based reasoning. Extracted features are transformed into queries against a datastore (PostgreSQL+pgvector) containing embedded threat intelligence documents. Relevant documents are retrieved using Haystack (Python library) and merged with the static features into a structured prompt. The prompt is then passed to an LLM such as GPT-4o, which produces a classification and supporting explanation. Details are given in Section 3.4.2.

3.2.5 Logging and Result Storage

Both pipelines record their outputs using a consistent schema with:

- Sample hash, timestamp, and pipeline identifier
- Classification label, score, and explanation
- Supporting metadata, latency, and reference sources

These logs provide the basis for reproducibility and comparative evaluation (Section 3.7).

3.3 Data Sources

The system relies on two categories of input: malware binaries for analysis, and external intelligence documents for retrieval. As shown in Figure 3.1, both pipelines begin with the same samples but diverge in how they use supporting data. The baseline reconciles static heuristics with VT tags, while the LLM+RAG pipeline augments static features with retrieved context.

3.3.1 Malware Samples Sourcing

The main input focuses on PE format files of five common malware categories found in systematic reviews [3, 1, 15]: **Trojan**, **Ransomware**, **Botnet**, **Worm**, and **Rootkit**. Three representative samples are drawn at random for each type (15 total) from VT and/or MalwareBazaar, and analyzed by both pipelines. To avoid biases, these samples's metadata were not ingested onto the RAG datastore.

The PE format file format was chosen for this research as it is the standard for Windows executables (e.g., `.exe`, `.dll`), is widely used in static Malware analysis due to its rich structural features such as headers, import/export tables, and embedded strings [6].

Each sample is catalogued with its SHA256 hash and timestamp. They are obtained from:

- **MalwareBazaar**: An open repository maintained by abuse.ch, providing recent PE malware binaries for research.
- **VT**: Accessed through its API, supplying binaries, scan metadata, and labeling tags.

3.3.2 Baseline Metadata Sources

The baseline analysis pipeline does not require external contextual data to work, however it uses a set of parameters that, when combined with the built in rules, define the thresholds and tune the sample analysis score and reconciliation strategy that fuses heuristics derived from Ghidra with consensus scores from VT. As can be seen at section 3.4.1.

3.3.3 RAG Corpus Data Sources and Construction

The LLM+RAG pipeline relies on a dedicated corpus of contextual documents to provide grounding during inference. This corpus was assembled through the ingestion process described in Section 3.4.3, and populated with high-quality open sources covering different levels of malware knowledge. Three main categories of data were included:

- **VX-Underground APT Writeups**: All Advanced Persistent Threat (APT) reports published between 2022 and August 2025 were downloaded and ingested. These reports contain adversary tactics, binary structures, and reverse engineering notes. The data were collected following project guides [1].
- **Malpedia Documentation**[19]: Family-level articles and technical descriptions were retrieved from Malpedia between 2022 and August 2025. These documents include IOCs, YARA rules, and summaries of static and behavioral traits. Some articles could not be included due to scraping failures, caused by protected pages, removed content, timeouts, or server errors (e.g., HTTP 503). Collection details are provided in the project guide [2].
- **MalwareBazaar Metadata**: Metadata records were retrieved through the MalwareBazaar API using batch queries. Tags such as **Trojan**, **Ransomware**,

[1] https://github.com/andremmfaria/rexis/blob/main/guides/DataSourcingVXUnderground.md, https://github.com/andremmfaria/rexis/blob/main/guides/IngestVXUnderground.md

[2] https://github.com/andremmfaria/rexis/blob/main/guides/DataSourcingMalpedia.md

Botnet, **Worm**, **Rootkit**, and **exe** were selected to align with the focus of this study on PE format-based malware. The metadata include sample hashes, tags, and antivirus results. Only metadata were ingested; raw binaries were excluded to keep the datastore limited to contextual evidence.

All documents were normalized, segmented, and embedded using the Haystack (Python library) framework. Each record was enriched with metadata fields such as source, document type, family or actor (if available), and timestamp. The processed chunks were then stored in a PostgreSQL+pgvector datastore for semantic retrieval during inference (Figure 3.2).

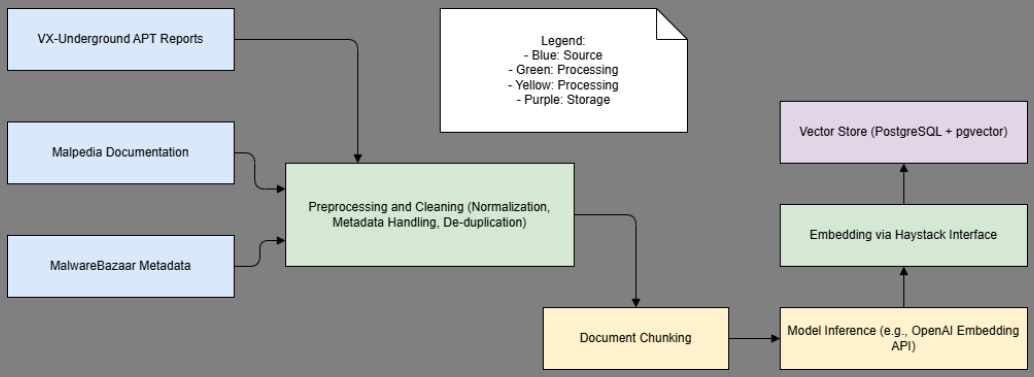

FIGURE 3.2: RAG Corpus Construction Pipeline

3.3.4 Labeling

Samples are annotated using metadata from VT when analysed through the baseline pipeline (see section 3.4.1). This provides a consistent benchmark for comparing the baseline and LLM+RAG pipelines in terms of accuracy and explanation quality.

3.4 Workflows

This section provides an explanation of the internal workings of each pipeline designed for the experiment. It explains in detail the steps involved in the baseline, LLM+RAG, and data ingestion pipelines.

3.4.1 Baseline Pipeline

The baseline pipeline provides a deterministic static classification process that does not rely on LLMs. It serves as a benchmark for comparison with the RAG-enhanced pipeline. The approach combines custom heuristics with VirusTotal metadata, using a reconciliation strategy to produce a single canonical label for each sample.

Input Format: The pipeline processes only PE formats, the native format for Windows executables. These files were chosen for their structural consistency and widespread use in malware distribution, which makes them well-suited for rule-based static analysis [6].

Feature Extraction: Static features are extracted from binaries using Ghidra with its Python API `PyGhidra`. The automated extraction step collects data based on Ghidra's automated binary detection engine.

These features (outlined on section 3.2.2) are extracted, serialized into JSON files for ease of extraction and usage.

Preprocessing and Logging: Extracted features are normalized and deduplicated, with all values logged for reproducibility. These logs form the structured input to the classification stage.

Hybrid Classification Strategy: Classification is performed by combining two sources of evidence:

- **Rule-Based Heuristics:** A lightweight decision engine evaluates static signals such as suspicious API calls (`CreateRemoteThread`, `VirtualAllocEx`), packed or encrypted sections, and known obfuscation markers. These rules produce heuristic labels such as "ransomware-like" or "downloader" [9, 5].
- **VirusTotal Metadata:** For each sample, the VirusTotal API is queried using the SHA-256 hash. Family names and vendor tags (e.g., `AgentTesla`, `Emotet`) are retrieved and normalized into canonical categories, reducing variation between vendors [22].

Reconciliation Logic: The final decision engine fuses heuristic output with VirusTotal metadata:

- If both sources agree, the label is accepted directly.
- If they diverge, the VirusTotal label takes precedence unless heuristic confidence surpasses a defined threshold.
- In ambiguous cases (e.g., low-confidence heuristics or missing VirusTotal tags), the decision engine applies weighted rules to produce a best-effort classification.

This weighted reconciliation balances determinism with practical reliance on external consensus. The full explanation on how the reconciliation engine works can be found on the project repository[3]. The user has the option to not include VirusTotal data on the pipeline. This makes the decision engine pass through the result from the heuristics engine effectively disabling this step.

Output Format: Each classification generates a structured report containing:

- **Canonical Family Label:** The reconciled family name (e.g., `LokiBot`), chosen after harmonizing vendor tags and heuristic results.
- **Classification Source:** Indicates whether the final decision was based on heuristics, VirusTotal tags, or a weighted combination of both.
- **Confidence Score:** A normalized value representing the strength of heuristic evidence, used when reconciling conflicting results.
- **Triggered Rules:** The specific heuristic rules that fired, providing transparency into why a heuristic label was assigned.
- **VirusTotal Matches:** The number of antivirus engines on VirusTotal that agreed on the family or type, offering a measure of external consensus.

[3] https://github.com/andremmfaria/rexis/blob/main/guides/Reconciliation.md

- **Metadata Snapshot:** Contextual information including file hash, timestamp, PE size, and section-level details, retained for audit and traceability.

The report structure is output in JSON format, enabling side-by-side comparison with the output of the RAG-based pipeline.

A detailed breakdown of the baseline pipeline design and decision logic is available in the guide: [4]

3.4.2 LLM+RAG Pipeline

The LLM+RAG pipeline integrates large language models with retrieval-augmented generation to extend the scope of static malware classification. Unlike the baseline, this approach uses semantic retrieval over a curated corpus of threat intelligence to provide context-aware inference and explanations. The method emphasizes inference-time reasoning instead of pre-trained classification logic, supporting zero-shot generalization without fine-tuning.

Input Format and Feature Extraction: The pipeline begins in the same way as the baseline one (see section 3.4.1). Each PE format sample is processed with Ghidra through `PyGhidra`, producing static features such as the ones seen in section 3.2.2. These features are logged into JSON format for use in later stages.

Semantic Query Generation: After feature extraction and preprocessing, the pipeline converts selected static attributes into a semantic query. Extracted values such as API imports, section entropy, and embedded strings are serialized into a structured textual representation. This representation is then embedded using OpenAI's `text-embedding-3-small` model, which produces dense vector representations suitable for semantic search. By translating low-level static features into natural-language-like descriptions, the system allows the retrieval engine to align binary characteristics with external threat intelligence documents. For example, a sample that imports `CreateRemoteThread`, `VirtualAllocEx`, and contains obfuscated networking strings would yield a query embedding that captures both behavioral and structural intent. The generated query vector is logged along with metadata such as chunk size, token count, and timestamp.

Contextual Document Retrieval: The embedded query is passed to a hybrid retriever implemented in Haystack (Python library). This retriever combines dense vector similarity search (via pgvector) with keyword-based BM25 retrieval, ensuring that both semantically similar and lexically overlapping documents can be surfaced. The underlying datastore is a PostgreSQL database enhanced with the `pgvector` extension, which holds embeddings and contents of curated malware related documents corpora (see 3.3.3). Documents are preprocessed into uniform chunks before embedding, with each chunk linked to metadata such as source type, family/actor tags, original filename, and publication date.

When a query is executed, the top-k candidate documents are returned along with their cosine similarity scores, source identifiers, and document IDs. If reranking is enabled (via a cross-encoder model such as `gpt-4o-mini`), the candidate set can be refined to prioritize documents with higher contextual relevance; by default, this step is skipped to reduce latency. All retrieval events are logged, including query latency, document ranks, and similarity values.

[4] https://github.com/andremmfaria/rexis/blob/main/guides/BaselinePipeline.md

Prompt Construction: Once the top-k documents are retrieved, they are combined with the extracted static features into The structured prompt template consists of the following key components:

- **Concise summary of static attributes:** Includes details such as API calls, entropy values, and extracted strings from the binary.
- **Retrieved contextual documents:** Presents the titles and relevant excerpts from the documents retrieved during semantic search.
- **Explicit instructions:** Directs the model to classify the sample, justify the decision with supporting evidence, and highlight related malware families if applicable.

Prompt construction is handled programmatically to ensure consistency across all samples. Sensitive content such as raw hex dumps, shellcode, or very large decompiled blocks is filtered out to mitigate prompt injection risks and to remain within token limits. Each generated prompt is logged with metadata including query ID, sample hash, document IDs, and token length, ensuring reproducible execution. For more information see section **??**.

LLM Inference and Response Handling: The completed prompt is submitted to GPT-4o via the OpenAI API to be used strictly in inference mode. Removing the need for fine-tuning, and emphasizing generalization from external context rather than learned weights. The LLM produces a structured response containing a proposed malware family label, justification using the retrieved evidence, and optional comparisons to related malware or tactics.

Response handling includes parsing the model output into structured fields (family label, rationale, references). Additional metadata such as model identifier, temperature setting, latency, and token usage are recorded. These details provide transparency in model performance and facilitate later analysis of cost, efficiency, and reproducibility.

Output Format: The output of the LLM+RAG pipeline is standardized to align with the baseline pipeline format while capturing additional context. Each classification record includes:

- **Sample identifiers:** SHA256 hash and timestamp of analysis
- **Predicted family label:** Canonicalized malware family or "unknown" if undecided
- **Supporting rationale:** Textual explanation generated by the LLM
- **Retrieved evidence:** List of document IDs, titles, source type (e.g., Malpedia, VX-U), and similarity scores
- **Model metadata:** Model name, API endpoint, token usage, and temperature setting
- **Performance metrics:** Latency measurements for embedding, retrieval, and inference

This uniform structure ensures comparability between the baseline and LLM+RAG pipelines while providing additional layers of interpretability and traceability. By explicitly linking each classification to both static features

and external documents, the pipeline produces outputs that are explainable, reproducible, and auditable.

A detailed breakdown of the LLM+RAG analysis pipeline design, implementation and prompt structure can be found in the guide: [5]

3.4.3 RAG Ingestion Pipeline

To support contextual retrieval in the RAG-enabled classification pipeline, a dedicated ingestion workflow was implemented to populate the vector datastore with structured, semantically rich documents. This pipeline is responsible for fetching, cleaning, chunking, embedding, and storing threat intelligence content, ensuring that LLMs are grounded in verified sources rather than generating unsupported inferences.

Input Data Sources: Three primary sources form the corpus:

- **VX-Underground APT Writeups** – APT campaign documentation containing attacker TTPs, reverse-engineering notes, and contextual indicators. Collected via curated archive scraping and exposed through a Python interface.
- **Malpedia Documentation** – Behavioral descriptions and malware family summaries, retrieved through Malpedia's API endpoints (e.g., references, bibliographies). These provide canonical mappings of families and actors.
- **MalwareBazaar Metadata** – Sample metadata, hashes, and tagging, retrieved via API and aligned with executable (e.g. `exe`) samples to match the scope of this project's PE format-focused analysis.

Data from all sources is collected in their original formats (Malpedia's JSON, VX-Undergrounds's PDFs, Malpedia's HTML or text, etc.), as described on 3.3.3.

Preprocessing and Cleaning: The ingestion pipeline applies normalization and filtering steps to maintain consistency across heterogeneous sources:

- Re-encoding to UTF-8 and removal of malformed characters
- Deduplication of repeated documents across feeds
- Removal of empty or trivial content blocks
- Uniform metadata enrichment, including document source, timestamp, malware family (if available), and threat category

This stage ensures downstream embeddings are consistent and traceable.

Chunking Strategy: Documents are split into smaller units to fit within embedding model limits[6] and to maximize retrieval granularity. The strategy is tuned per document type:

- **Prose (reports, articles)** – Maximum 500 words per chunk with 50-word overlap, sentence-boundary aware
- **Structured JSON/metadata** – Maximum 4000 characters with 400-character overlap, split by character range rather than sentence

[5] https://github.com/andremmfaria/rexis/blob/main/guides/LLMRagPipeline.md
[6] https://platform.openai.com/docs/guides/embeddings#embedding-models

- **Token-boundary fallback** – If a chunk exceeds the model's token limit (default 8192), it is split into smaller token batches without overlap

This strategy was required for the ingestion of documents of varying lengths and formats. The chunk sizes per format were chosen based on statistical evaluation based on document density and information richness [7].

Embedding and Vectorization: Prepared chunks are embedded using OpenAI's `text-embedding-3-small` model. Each chunk is converted into a high-dimensional vector and paired with its metadata (source, family, timestamp, embedding version). Embedding metadata is retained to allow model versioning and reproducibility across experiments.

Database Insertion: Chunks and embeddings are stored in PostgreSQL with the `pgvector` extension, enabling efficient cosine similarity search. Integration with Haystack handles bulk insertion, index construction, and metadata filtering. Each row in the datastore contains:

- Vector embedding
- Original text chunk
- Source metadata (filename, family/actor, source URL)
- Embedding version and timestamp
- Chunk indexing (position and total chunks for rejoining if needed)

This structure allows both fine-grained retrieval and reconstruction of the original document if required.

Full technical details on the ingestion workflow used to build the RAG corpus and scripts are provided in the guide: [8]

3.5 Prompt Engineering

The design of prompts is central to the performance of the LLM+RAG pipeline. Prompt construction influences both classification accuracy and the interpretability of results, as highlighted in prior studies on prompt engineering for reasoning tasks [20, 8]. To balance flexibility with reproducibility, prompts in this study follow a hybrid structure: a fixed system message that enforces output schema and consistency, and a dynamically assembled user message that integrates static features and retrieved evidence in a structured natural-language format.

3.5.1 Prompt Structure

Prompts follow a modular template with two main components:

- **System Message:** Defines the role of the model and enforces a strict JSON schema for outputs. Mandatory fields include family label, classification type, capabilities, tactics, supporting evidence, and an uncertainty measure. By fixing this schema at the system level, structural consistency is maintained across all samples.

[7] https://learn.microsoft.com/en-us/azure/search/vector-search-how-to-chunk-documents
[8] https://github.com/andremmfaria/rexis/blob/main/guides/IngestionPipeline.md

- **User Message:** Dynamically assembled per sample, divided into three sections:

    1. **Context Block:** A concise list of static features extracted from the binary (e.g., API imports, entropy values of sections, embedded strings), formatted as natural-language bullet points.
    2. **Retrieved Documents:** Up to three semantically similar documents drawn from the vector store. Each entry includes its source (e.g., Malpedia, VX-Underground), title or tag, document ID, and a short snippet.
    3. **Task Instructions:** A fixed directive asking the model to classify the sample into a known family and justify the decision with evidence, or compare it with related malware if applicable. This component varies by task type (see Section 3.5.3).

3.5.2 Filtering and Safety

Before submission to the model, prompts are filtered to remove irrelevant or unsafe content. Large hex dumps, decompiled code blocks, and sensitive network indicators are excluded to reduce noise and mitigate prompt-injection risks. This preprocessing step complements the guardrail strategies described in Section 3.6, which apply downstream safeguards such as evidence validation and abstention.

3.5.3 Task Templates

Three prompt variants are implemented to cover different reasoning requirements:

1. **Classification-Only:** Requests a family label based on static features and retrieved context.
2. **Classification + Justification:** Requires the model to provide a label *and* cite both static features and retrieved evidence to support its decision.
3. **Classification + Comparison:** Adds a comparative element, asking the model to relate the sample to known families or behaviors, highlighting similarities and differences.

These templates allow systematic observation of how the model handles increasing reasoning demands while maintaining schema stability.

3.5.4 Reproducibility and Versioning

Every prompt is versioned using a hash computed from its static features and retrieved document IDs. This ensures that any model output can be traced back and reproduced, even if the underlying datastore or APIs are later updated. Prompt versions and inference logs are stored alongside classification results, providing traceability for evaluation (see Section 3.7).

3.5.5 Prompt Example

The excerpt below illustrates how prompts are assembled in the LLM+RAG pipeline. The system message enforces schema compliance, while the user message provides

structured context, retrieved evidence, and task instructions. Only the user message is shown for clarity.

Prompt

```
Context:
- The file imports CreateRemoteThread, VirtualAllocEx, and
WriteProcessMemory.
- High entropy detected in .text and .rsrc sections.
- Embedded strings include URLs (hxxp://malicious-site.com) and encoded
PowerShell commands.

Retrieved Document [Malpedia: AgentTesla, DocID=malp-1234]:
"AgentTesla often uses process injection via RemoteThread APIs and
obfuscates its communication channels with encoded PowerShell payloads."

Retrieved Document [VX-Underground: APT37, DocID=vx-5678]:
"APT37 campaigns leveraged DLL injection combined with custom packers,
embedding URLs for remote payload retrieval."

Task:
Based on the static features and retrieved documents, classify the
sample into a known family. Provide a JSON output following the schema,
including:
-- family label
-- classification tags (e.g., trojan, stealer)
-- capabilities and tactics
-- 4-8 evidence items citing features and retrieved documents
-- an uncertainty measure (low, medium, high)

If uncertain, return "label": "unknown" and "uncertainty": "high".
```

Prompt design directly impacts interpretability, since structured schemas and explicit evidence citation allow explanations to be measured against the criteria defined in section 3.7.

3.6 Guardrails

To mitigate the risks of inaccurate or wrongful classification in the LLM+RAG pipeline, this study introduces a set of *guardrail strategies* that improve reliability, factual grounding, and resilience to contextual bias. Guardrails are defined here as both *technical controls* (e.g., input sanitization, evidence validation) and *procedural measures* (e.g., audit trails, usage controls, human oversight) designed to ensure safe, ethical, and reproducible operation. These strategies complement prompt engineering (see Section 3.5), which governs how inputs are presented, by adding downstream mechanisms that enforce trustworthy outputs.

3.6.1 Rationale and Basis

RAG improves factual grounding by incorporating external documents, but hallucinations still arise from retrieval noise, lexical anchoring, or overreliance on spurious cues such as family names [23]. Structured prompting and multi-stage reasoning (e.g., hypothesize then verify) reduce hallucination and improve reliability [4, 12]. In parallel, system-level *guardrails* that sanitize inputs, enforce

verification, and log outputs have been shown to enhance trustworthiness in LLM deployments [11, 14].

3.6.2 Guardrail Components

1. **Name Redaction in Retrieved Passages.** Masking known family names or actor terms in retrieved context helps avoid anchoring bias. This aligns with input sanitization techniques advocated in safety-critical LLM systems [11].

2. **Re-Rank by Technical Evidence.** Retrieved passages are reordered based on domain-specific technical cues (e.g., API calls, section labels) while down-weighting passages that emphasize family names. This improves factual relevance and reduces susceptibility to lexical noise [23, 14].

3. **Two-Stage Classification: Blind Hypothesis → Evidence Verification.** A classification hypothesis is first generated from structured features alone, then validated against sanitized RAG context. This structured verification encourages evidence-based reasoning [4, 12].

4. **Evidence Threshold & Abstention.** If fewer than two retrieval-based evidence snippets support a claim, or if the proposed family matches a redacted term, the system abstains by returning `"unknown_family"`. This prevents overconfident misclassification [11, 14].

5. **Transparent Audit Trail.** Each classification records redacted names, reranked passages, blind vs. final hypotheses, evidence counts, and abstention status. [11].

3.6.3 Workflow Summary

Step	Description
1. Redaction	Remove family/actor names from retrieved texts
2. Re-Ranking	Score passages by technical relevance, penalize names
3a. Blind Classification	Hypothesize using structured features only
3b. Evidence Verification	Validate hypothesis using sanitized RAG
4. Evidence Threshold & Abstain	Require $\geq$2 quotes; abstain if insufficient
5. Audit Trail	Record metadata for analysis and trustworthiness
6. Human Oversight	Flag ambiguous or critical cases for review

3.6.4 Theoretical Justification

- *RAG grounding* reduces hallucination but requires reinforcement to ensure fidelity [23, 4].
- *Structured verification (hypothesis + evidence)* encourages factual reasoning and curbs overconfident outputs [4, 12].
- *System-level guardrails*, including sanitization, abstention, and audit trails, are critical for trust in deployed LLM systems [11, 14].
- *Ethical oversight*, through human-in-the-loop review of ambiguous cases, aligns the pipeline with responsible AI practices and prevents misuse in operational cybersecurity settings.

3.7 Evaluation Framework

This section defines the framework used to evaluate both the baseline and RAG-enhanced pipelines. Evaluation is organized around three categories: classification accuracy, processing efficiency, and interpretability of results. These criteria are selected to balance quantitative performance with qualitative insight, reflecting practical requirements of malware analysis in operational contexts. The framework ensures that results are directly comparable across pipelines by using a unified schema for classification outputs.

Limitations of this framework, including reliance on vendor consensus and the absence of human annotation, are discussed in Section 3.8.

3.7.1 Accuracy

Classification accuracy measures the proportion of correctly assigned categories relative to VirusTotal consensus ground truth:

$$\text{Accuracy} = \frac{N_{\text{correct}}}{N_{\text{total}}} \tag{3.1}$$

where N_{correct} is the number of samples whose predicted categories match the benchmark label, and N_{total} is the total number of samples evaluated. To ensure comparability, all ground truth labels are normalized into canonical category names. This step mitigates inconsistencies between vendor taxonomies and ensures that both pipelines are scored against the same reference.

3.7.2 Efficiency

Efficiency captures average processing latency per sample:

$$\text{Efficiency} = \frac{1}{N_{\text{total}}} \sum_{i=1}^{N_{\text{total}}} T_i \tag{3.2}$$

where T_i denotes end-to-end processing time for sample i. For the LLM+RAG pipeline, this includes embedding generation, document retrieval, prompt assembly, and model inference latency. For the baseline, T_i encompasses decompilation, feature extraction, rule evaluation, and enrichment via external APIs. This metric highlights trade-offs between richer semantic reasoning and the computational cost of achieving it.

3.7.3 Interpretability

Interpretability is assessed using three sub-metrics adapted from Wong *et al.* [22] and Sahoo *et al.* [20]. These measures capture both the presence and quality of explanations:

1. **Explanation Coverage:**

$$\text{Coverage} = \frac{N_{\text{explained}}}{N_{\text{total}}} \tag{3.3}$$

where $N_{\text{explained}}$ is the number of samples accompanied by an explanation.

2. **Contextual Grounding:**

$$\text{Grounding} = \frac{N_{\text{with-citation}}}{N_{\text{explained}}} \tag{3.4}$$

   where $N_{\text{with-citation}}$ is the number of explanations citing at least one retrieved document, ensuring justification is evidence-based rather than speculative.

3. **Rule-Reference Matching (Baseline only):**

$$\text{RuleMatch} = \frac{N_{\text{consistent-rules}}}{N_{\text{triggered-rules}}} \tag{3.5}$$

   where $N_{\text{consistent-rules}}$ is the number of heuristic rules whose triggers are consistent with the final family label. This provides an analogue to justification in the baseline pipeline.

These sub-metrics allow interpretability to be assessed not only by output presence, but also by the degree to which explanations are grounded in verifiable evidence.

3.7.4 Composite Score

To enable a holistic comparison between pipelines, a composite score is defined as a weighted sum of the three axes:

$$\text{Composite} = \alpha \cdot \text{Accuracy} + \beta \cdot \text{Efficiency}^{-1} + \gamma \cdot \text{Interpretability} \tag{3.6}$$

where $\alpha + \beta + \gamma = 1$. Efficiency is inverted so that lower processing times contribute positively. By default, equal weights are applied ($\alpha = \beta = \gamma = \frac{1}{3}$), though weights can be adjusted depending on analytic priorities (e.g., prioritizing accuracy over efficiency in high-stakes detection scenarios).

3.8 Reliability, Validity, and Limitations

This section outlines the measures taken to ensure reproducibility of the experiments, discusses internal and external validity considerations, and presents the primary limitations of the study. Together, these factors frame the boundaries of interpretation for the evaluation results presented in Chapter 3.7.

3.8.1 Reproducibility

Reproducibility is supported through a combination of methodological rigor and technical infrastructure:

- **Version-Controlled Codebase**: All pipeline code, configuration files, and documentation are maintained in a public GitHub repository.[9]
- **Containerized Environment**: All software dependencies are managed via `PDM`, deployed in Docker-based containers or are installed in isolated directories.

[9] https://github.com/andremmfaria/rexis

- **Explicit API Versions**: External services such as OpenAI, VirusTotal, and Malwarebazaar are accessed through versioned API endpoints, with documented model identifiers (e.g., `gpt-4o`, `vt-api`).
- **Metadata Logging**: Each pipeline run produces structured logs, recording sample identifiers, timestamps, toolchain versions, and model metadata.

3.8.2 Internal Validity

Internal validity is maintained by controlling experimental variables and applying consistent processing steps across pipelines. Static features are extracted using a shared Ghidra backend, while retrieval, embedding, and prompt templates remain fixed throughout experiments. No manual analyst intervention is introduced into classification workflows, avoiding bias.

Potential threats include:

- **Heuristic Bias**: The baseline pipeline's rule-based logic may overfit to common rules and patterns and fail against obfuscation or novel attack techniques. Although the rule set can be expanded, coverage gaps remain.
- **Retrieval Drift**: Embedding-based retrieval is sensitive to corpus changes and chunking strategies, which may influence which context is provided to the model. This is partially mitigated by retrieval guardrails (Section 3.6) and prompt versioning (Section 3.5), but residual variability may still affect consistency across runs.

3.8.3 External Validity

Findings are limited in scope to PE format-based malware and static analysis scenarios. Results may not generalize to fileless malware, non-Windows binaries, or dynamic behavior-based classification. Inference-only LLMs are employed without fine-tuning. This choice enhances reproducibility and reduces operational cost, but it limits adaptability to analyst-in-the-loop scenarios where domain-specific customization would normally be expected. By excluding hybrid or dynamic analysis features, the results may not generalize to systems that integrate runtime behavior with static evidence, specially on obfuscated malware.

3.8.4 Evaluation Limitations

The evaluation framework described in Section 3.7 has several constraints that affect how results should be interpreted.

- **Ground Truth Reliance**: Dependence on VirusTotal consensus introduces noise and inconsistency, which directly impacts the reliability of accuracy scores.
- **Human-in-the-Loop Absence**: The lack of expert annotation or analyst feedback limits the validity of interpretability assessments.
- **Surface-Level Metrics**: Metrics such as coverage and citation capture structural qualities but not the semantic fidelity of explanations, potentially overstating interpretability performance.

These constraints affect the precision of comparative evaluation and highlight that interpretability scores should be interpreted cautiously.

3.8.5 Study Limitations

The study operates under several broader constraints that limit generalizability:

- **Static Analysis Only**: No dynamic or hybrid analysis is performed. Obfuscated or packed binaries may yield incomplete or misleading feature sets.
- **LLM Limitations**: Generated responses may contain hallucinations or overconfident statements [24, 7]. While prompts are structured to minimize risk, adversarial testing is not included.
- **Dataset Scale**: Sample sizes are constrained by API quotas and curation effort, which restricts statistical robustness. This also introduces potential class imbalance, where common families are overrepresented and rare or emerging families receive insufficient coverage.
- **No Adversarial Robustness**: The system is not evaluated against adversarial risks such as prompt injection, retrieval poisoning, or obfuscation-specific evasion. These are well-documented threats to RAG-powered pipelines [24], and their omission limits claims about robustness under hostile conditions.
- **Ethical limitations**: Automated classification carries risks of false positives and mislabeling, which in real-world contexts could affect operational decisions or intelligence reporting. While this study mitigates some risks through audit trails and guardrails (Section 3.6), ultimate responsibility for critical classifications rests with human analysts.

Taken together, these limitations emphasize that while the study provides a reproducible and systematic framework for evaluating LLM+RAG pipelines in static malware analysis, its conclusions should be interpreted within a bounded scope. Technical constraints shape the reliability of results, dataset and evaluation limitations affect generalizability, and ethical considerations highlight the need for human oversight in operational deployments. These caveats do not undermine the validity of the findings, but they define the context in which they can be responsibly applied.

3.9 Ethical Considerations

The research presented in this study involves the use of malware samples, static code artifacts, and third-party threat intelligence sources. Several precautions are taken to ensure that the study adheres to ethical standards in cybersecurity research.

3.9.1 Safe Handling of Malware Samples

All malware samples are statically analyzed without execution. No sandboxing or dynamic behavior monitoring is performed. Samples are stored in a secure, containerized, non-networked, isolated environment with no file execution permissions. The analysis pipeline operates solely on extracted features (e.g., strings, imports, entropy) and decompiled code sections.

Samples are sourced from public research repositories such as MalwareBazaar and VirusTotal, which provide malware binaries and metadata specifically for research and detection engineering purposes. No private or unknown-origin malware is introduced into the dataset.

3.9.2 Use of Public and Licensed APIs

All external APIs (OpenAI, VirusTotal, MalwareBazaar) are accessed through official channels with proper authentication and rate limits. API usage complies with the respective Terms of Service, and no bulk data scraping or unauthorized reverse engineering is conducted.

VirusTotal's API is used strictly for metadata enrichment and tagging. No personal data, user-uploaded files, or proprietary content is accessed or stored.

3.9.3 Privacy and Data Anonymity

No personal, user-identifiable, or system-specific data is involved in the research. All data used is either public (e.g., APT reports) or anonymized (e.g., sample hashes). Retrieved documents are cleaned to remove IP addresses, hostnames, or other sensitive references prior to embedding.

3.9.4 Responsible Use of LLMs

The use of LLMs in this study is limited to inference-only interactions. No fine-tuning is performed. The prompts are explicitly filtered to exclude shellcode, network addresses, or exploit payloads. The system is designed for classification and explanation, not for generating or manipulating malware.

To prevent propagation of misleading or unsafe content, all generated responses are stored and reviewed offline. In alignment with recommendations from recent LLM safety surveys [7, 16], the system avoids code synthesis and adheres to a read-only analysis posture.

3.10 Summary

This chapter established the methodological framework guiding the evaluation of retrieval-augmented LLMs in static malware analysis. The research design (Section 3.1) follows an experimental, prototype-based approach that contrasts a baseline classifier—combining static heuristics with VirusTotal metadata—against an enhanced LLM+RAG pipeline. The system architecture (Section 3.2) and experimental workflow (Section 3.1.1) were presented in detail, showing how Ghidra-based static feature extraction, containerized environments, and a unified ingestion process are combined to provide consistent and reproducible analysis.

Guardrail strategies (Section 3.6) and prompt engineering methods (Section 3.5) were introduced as core mechanisms for ensuring safety, accountability, and interpretability. These include name redaction, evidence thresholds with abstention, transparent audit trails, and fixed prompt templates. The role of data sources (Section 3.3) was described in relation to Malpedia, MalwareBazaar, and VX-Underground, which populate the retrieval corpus that grounds LLM reasoning. Together with reproducibility measures (Section 3.8.1), these strategies mitigate risks such as hallucination, anchoring bias, and retrieval drift.

Finally, the evaluation framework (Section 3.7) was defined across three axes: accuracy, efficiency, and interpretability, with a composite score to enable holistic comparison. Reliability and validity considerations (Section 3.8) were addressed, and key limitations acknowledged, including reliance on VirusTotal consensus, dataset imbalance, the absence of adversarial robustness testing, and ethical concerns surrounding automated classification. Together, these elements ensure

that the methodology directly addresses the research questions of this thesis while clearly delimiting the scope for interpretation. The next chapter presents the implementation of these pipelines and the results of their evaluation.

Chapter 4

Implementation and Results

This chapter presents the implementation details and key findings of the research system designed to enhance static malware analysis using LLMs and RAG. It outlines the architectural realization of the proposed pipelines, describes how individual components were integrated, and evaluates their performance across a representative corpus of malicious PE formats samples. The experiments were conducted on a dataset of 15 malware samples across five commonly studied families [3, 1, 15] (See section 3.3.1), enabling comparative evaluation of classification behavior.

4.1 Environment

The experimental environment was built using a combination of open-source tools, APIs, and containerized services, with an emphasis on modularity, reproducibility, and transparent workflows. All development was carried out in Python, with dependencies managed via isolated environments to ensure deterministic builds. A step-by-step guide to the setup and configuration can be found in the project repository.[1]

The main tools and frameworks are:

- **Python** – Primary implementation language for orchestration logic and pipeline components. Python was chosen for its mature ecosystem in cybersecurity, data analysis, and Machine Learning.[2]
- **PDM (Python Development Master)** – Manages dependencies and virtual environments. PDM provides deterministic builds and isolated execution, supporting reproducibility across runs and machines.[3]
- **PostgreSQL + pgvector** – Relational database extended with the extttpgvector module[4][5] for similarity search on embeddings. It serves as the vector store backing the retrieval stage of the RAG pipeline. The database is deployed in a Docker container for consistency and portability.[6]
- **Haystack (Python library)** – Framework used to implement the RAG workflow, including ingestion, chunking, embedding, and retrieval. Its

[1] https://github.com/andremmfaria/rexis
[2] https://www.python.org
[3] https://pdm-project.org
[4] https://www.postgresql.org
[5] https://github.com/pgvector/pgvector
[6] https://www.docker.com

modular components simplify orchestration and integration with external LLMs.[7]

- **GPT-4o (inference-only)** – Used strictly in inference-only mode. No fine-tuning was performed, ensuring reproducibility across runs. The model is responsible for contextual reasoning and static feature analysis, driven by structured prompts enriched with retrieved evidence.[8]
- **Ghidra + PyGhidra** – Ghidra is used as the static analysis and decompilation framework. PyGhidra provides bindings to automate extraction of imports, functions, strings, sections, entropy values, and decompiled code from PE format binaries, enabling feature logging at scale.[9][10]
- **Docker** – Used to containerize core services such as PostgreSQL with pgvector, the Ghidra server, and auxiliary tools. Containerization ensures reproducibility, isolation, and consistency across systems.[11]
- **MalwareBazaar API** – Provides malware samples, hashes, tags, and metadata, serving as a key source of enrichment and validation.[12]
- **VirusTotal API** – Supplies scan metadata, antivirus tags, and consensus labels across vendors. These labels support the baseline classifier and reconciliation logic, providing a benchmark for evaluation.[13]
- **VX-underground and Malpedia** – Additional data sources used to populate the retrieval corpus with threat intelligence reports, APT writeups, and malware family descriptions. These documents strengthen contextual grounding in the RAG pipeline by supplying semantically rich background evidence.[14][15]

Version control was managed through Git and GitHub, ensuring traceability of code, configurations, and experiment scripts. Each pipeline execution was logged with structured metadata, including sample identifiers, timestamps, tool versions, prompt hashes, and retrieved document IDs, supporting full auditability and reproducibility. Containerization and PDM further guaranteed that results can be replicated on any compliant environment with minimal configuration drift.

4.2 Implementation Overview

The system is implemented as a modular Python application structured around the `Typer`[16] CLI framework. The codebase (`src/rexis`) is organized to support three main operations: data ingestion, baseline static classification, and enhanced LLM+RAG classification. Common modules handle configuration, logging,

[7] https://haystack.deepset.ai
[8] https://platform.openai.com/docs/api-reference/introduction
[9] https://github.com/NationalSecurityAgency/ghidra
[10] https://github.com/NationalSecurityAgency/ghidra/blob/master/Ghidra/Features/PyGhidra/src/main/py
[11] https://www.docker.com
[12] https://bazaar.abuse.ch/api/
[13] https://developers.virustotal.com/reference
[14] https://vx-underground.org
[15] https://malpedia.caad.fkie.fraunhofer.de
[16] https://typer.tiangolo.com/

and feature extraction, while each pipeline defines its own classification logic. Documentation and usage examples for each pipeline are provided in dedicated guides: ingestion[17], baseline[18], and LLM+RAG[19].

The code project can be found at `https://github.com/andremmfaria/rexis`.

4.2.1 Common Modules

The following components are shared across all pipelines:

- **Sample Loader:** Reads PE format binaries, computes hashes (SHA256, MD5), and registers samples in a consistent directory layout for reproducibility.
- **Ghidra Integration:** Static feature extraction is automated via Ghidra using the `PyGhidra` API. Features include imports, functions, printable strings, section entropy, entry points, and decompiled code.
- **Feature Extractor:** Serializes features into JSON and `pandas` DataFrames for consistency and downstream reuse.
- **Logger:** Structured logging captures runtime events, timestamps, tool versions, hashes, and prompt identifiers. This provides an audit trail for every classification decision.
- **Configuration Manager:** Runtime parameters such as API keys, thresholds, and feature toggles are handled by `dynaconf`[20], ensuring secrets are separated from code and allowing profile-specific overrides.
- **CLI Interface:** A unified `rexis` command provides access to all pipelines. Subcommands support ingestion (`rexis ingest`), baseline classification (`rexis run baseline`), and RAG-enhanced classification (`rexis run rag`).

4.2.2 Data Ingestion Pipeline

The ingestion pipeline prepares the retrieval corpus by transforming raw threat intelligence into searchable vector embeddings:

- **Document Loader:** Collects documents from MalwareBazaar, Malpedia, and VX-Underground, normalizing metadata and assigning unique IDs.
- **Preprocessing Engine:** Cleans input by removing boilerplate, scripts, and noise, standardizing text for downstream processing.
- **Chunker:** Splits long documents into sections sized for embedding-model token limits using heading- and length-aware heuristics.
- **Embedder:** Encodes chunks using OpenAI's `text-embedding-3-small` model. Embeddings are processed in batches for scalability.

[17] `https://github.com/andremmfaria/rexis/raw/refs/heads/main/guides/IngestionPipeline.md`

[18] `https://github.com/andremmfaria/rexis/raw/refs/heads/main/guides/BaselinePipeline.md`

[19] `https://github.com/andremmfaria/rexis/raw/refs/heads/main/guides/LLMRagPipeline.md`

[20] `https://www.dynaconf.com/`

- **Vector Store Writer:** Stores embeddings and metadata in a PostgreSQL+pgvector database. Haystack's `DocumentStore` abstraction is used for indexing and similarity search.

Ingestion is automated via `rexis ingest -source=<name>` and supports incremental updates. Prompt versioning ensures that classification results can always be traced to the exact retrieval corpus state.

4.2.3 Baseline Pipeline Implementation

The baseline pipeline combines heuristic rules with VirusTotal metadata to provide a deterministic classification strategy:

- **Heuristic Classifier:** Applies manually curated rules to extracted features. For example, suspicious imports or anomalous entropy can trigger alerts.
- **VirusTotal Enricher:** Queries the VirusTotal API for detections, tags, and metadata. Results are normalized to canonical family labels via a tag-mapping layer.
- **Decision Engine:** Reconciles heuristic outputs with VirusTotal consensus using confidence-weighted thresholds.
- **Output Formatter:** Produces structured JSON summaries containing rules triggered, VirusTotal consensus, and final classification label.

4.2.4 LLM+RAG Pipeline Implementation

The enhanced pipeline retrieves semantically relevant evidence and integrates it into structured prompts for inference via GPT-4o:

- **Semantic Query Builder:** Transforms extracted features into natural language queries, incorporating imports, strings, and header attributes.
- **Retriever:** Executes top-k retrieval from the pgvector store using Haystack, returning documents with IDs, scores, and snippets.
- **Prompt Builder:** Assembles prompts from static features, retrieved passages, and task instructions. Prompt schemas are fixed to ensure consistent outputs (see Section 3.5).
- **Compliance Filter:** Sanitizes prompts by removing hex dumps, sensitive domains, or oversized content, reducing injection risks (see Section 3.6).
- **Inference Client:** Submits prompts to GPT-4o via the OpenAI API, logging model ID, latency, and token counts for reproducibility.
- **Result Parser:** Extracts structured fields (family, classification tags, rationale, citations, uncertainty) from model outputs, aligning them with the evaluation framework (Section 3.7).

4.3 Summary of Key Findings

TODO...

4.4 Interpretation of Findings

TODO...

4.5 Implications of the Study

TODO...

4.6 Limitations

TODO...

4.7 Recommendations for Future Research

TODO...

4.8 Conclusion

TODO...

Chapter 5

Conclusion

TODO...

5.1 Summary of Research

TODO...

5.2 Answers to Research Questions

TODO...

5.3 Research Contributions

TODO...

5.4 Implications of the Study

TODO...

5.5 Limitations

TODO...

5.6 Recommendations for Future Research

TODO...

5.7 Final Reflections

TODO...

Appendix A

Frequently Asked Questions

A.1 Getting Feedback

1. Get feedback **often** and from different audiences – your family, friends, professors, colleagues, advisor, other graduate students. The more you talk about your research, the more comfortable you get with it.

2. Keep a positive attitude. Research is hard. If it were easy, everyone would be doing it.

3. Consider setting up or joining a thesis group to share your ideas and experiences.

Supervisor's feedback

Some supervisors will ask for you to send each chapter as you complete it, offering feedback at that point, and then again at the end when the thesis chapters are collated. Other supervisors may want to be more involved, and there are others who will not want to see your thesis until it is completed by your standards. Whichever approach your supervisor takes, be aware that they will need some time to read through your work and provide feedback. Your thesis review is likely not the only piece of work your supervisor is undertaking, so be patient and factor review time into your work schedule.

A couple of points to note about supervisor feedback:

- You will receive feedback on your approach to research (i.e., method, experiment design etc...) as well as your writing. It is your responsibility to take notes at meetings etc. in order to record this feedback. It is also up to you whether or not you act upon the feedback provided.

- Your supervisor's role is to guide your work. It is not their job to complete the research, suggest methods, design experiments, or to write/rewrite sections of your thesis.

- Your supervisor should be supportive but sometimes their feedback may be difficult to hear. Just remember, their goal is to guide you and to make you a better researcher. Learn to have your work criticised in a constructive manner, it is part of the learning process.

- It is not the role of your supervisor to proofread your thesis. Many supervisors will point out typos, grammatical errors or styling issues etc. when they see them, but this is not their role.

A.2 Proofreading/copyediting

It is important to have your work proofread[1]. If English is not your first language, this is even more important for you.

How?

A good approach is to proofread yourself as you write and then again when you are finished writing a section or chapter. When you have a near final draft, have it proofread by a friend, family member, colleague, or a classmate etc... (not your supervisor). Choose your proofreader wisely. Make sure that they have good written English skills and are able to spot grammatical errors. A native English speaker can be good for this but not all native English speakers have the skills needed to be a good proofreader.

There are many things to look out for when reviewing your own work, everything from text alignment and section numbering, to figures and tables, to spelling and grammar. It's best to identify and fix any of these errors immediately. Don't wait until the end because these will build up and it often takes longer than you think to fix them.

If you find that you make the same mistake regularly, e.g., you misspell the same word regularly, or you use a colon where you shouldn't, then make a list of these to check back when you are finished each section (the search feature is good for this).

[1] Two types of editing that are commonly used interchangeably are copy editing and proofreading. Both types of editing clean up writing, but each has its distinct contribution to the process. https://thesiswhisperer.com/2016/11/30/doing-a-copy-edit-of-your-thesis/

A.3 Writing Assistants

In the past, students may have used tools such as Grammarly or Quetext, but this has become more problematic because such editing tools now come with AI assisted writing (see more here: https://tudublin.libguides.com/c.php?g=720901&p=5233062).

Using tools such as a spell checker, a grammar checker, and a punctuation checker are generally acceptable. Using more advanced tools to rewrite sentences, check tone, offer alternative word choices, offer citations etc... is not acceptable.

If in doubt don't use any such software. In general, it appears that, as of 2025, the free version of Grammarly is fine to use, but the pro version is not.

A.4 Example of Longtable

Ticket Type ID	Description
300	Feeder Ticket - Child
301	Feeder Ticket - Adult
310	10-Journey Feeder - Adult
317	Airlink Adult Airport-Busarus
318	Airlink Child Airport-Busarus
319	Airlink Child Airport-Heuston
320	Airlink Adult Airport-Heuston
333	Adult Single Feeder
365	Child Bus/Rail Short Hop - Day
366	Adult Bus/Rail Short Hop - Day
367	Family Bus/Rail Short Hop - Day
369	4 Day Explorer
410	Weekly Adult Short Hop Bus/Rail
430	Weekly Adult Medium Hop Bus/Rail
431	Weekly Adult Long Hop Bus/Rail
432	Weekly Adult Giant Hop Bus/Rail
433	Monthly Adult Short Hop Bus/Rail
455	Monthly Adult Long Hop Bus/Rail
456	Monthly Adult Giant Hop Bus/Rail
457	Monthly Student Short Hop Bus/Rail
458	Annual Bus/Rail
478	Annual All CIE Services
479	Annual CIE Pensioner Bus/Rail
480	Monthly CIE Pensioner Bus/Rail
493	Foreign Student - 1 Week
494	Foreign Student - 2 Week
495	Foreign Student - 3 Week
496	Foreign Student - 4 Week
497	CYC Group
600	Adult Cash Fare
608	Nitelink (Maynouth/Celbridge)
609	Nitelink (Maynouth/Celbridge)
610	Child Cash Fare
620	Schoolchild Cash Fare
625	Adult (formerly Shopper)
630	Adult 10-Journey (3 Stages)
631	Adult 10-Journey (7 Stages)
632	Adult 10-Journey (12 Stages)
633	Adult 10-Journey (23 Stages)
634	Adult 10-Journey (23+ Stages)
640	Adult 2-Journey (3 Stages)
641	Adult 2-Journey (7 Stages)
642	Adult 2-Journey (12 Stages)
643	Adult 2-Journey (23 Stages)
644	Adult 2-Journey (23+ Stages)
650	Schoolchild 10-Journey
651	Scholar 10-Journey
652	Schoolchild 2-Journey
653	Scholar 2-Journey
657	Transfer 90 (or Passenger Change)
658	Adult Single Heuston-CC
660	Adult One Day Travelwide
661	Child One Day Travelwide
662	Family One Day Travelwide
665	Rambler (3 Day Bus only)
670	Weekly Adult Bus

Ticket Type ID	Description
671	Weekly Adult Cityzone
690	Weekly Student Travelwide
691	Weekly Student Cityzone
705	Monthly Adult Citizone (AerLingus.)
710	Monthly Adult Travelwide
730	Annual Adult Travelwide
760	Annual Staff Bus
790	School Pass
791	OAP Pass
800	City Tour - Adult
801	City Tour - Family
802	City Tour - Child
898	10 - Journey Test Ticket

Glossary

AI systems See Artificial Intelligence

Artificial Intelligence The simulation of human intelligence in machines designed to think, learn, and solve problems

machine intelligence See Artificial Intelligence

API See Application Programming Interface

Application Programming Interface A set of protocols and tools that allows different software applications to communicate and interact with each other

LLMs See Large Language Model

language model See Large Language Model

Large Language Model A type of AI trained on vast text data to understand, generate, and manipulate human language

GPT-4o A multimodal Large Language Model developed by OpenAI that processes text, vision, and audio inputs natively and in real time. See https://openai.com/blog/gpt-4o

DeepSeek-R1 An open-source Large Language Model developed by DeepSeek, designed for general-purpose reasoning and code understanding. See https://deepseek.com/research

Chat-GPT See ChatGPT

Chat GPT See ChatGPT

ChatGPT An AI language model developed by OpenAI that can understand and generate human-like text based on prompts. See https://chat.openai.com

Machine Learning A subset of artificial intelligence that enables systems to learn and improve from experience without being explicitly programmed

ML model See Machine Learning

ML system See Machine Learning

malicious software See Malware

Malware Malicious software designed to disrupt, damage, or gain unauthorized access to computer systems

malware sample See Malware

static analysis See Static Malware Analysis

Static Malware Analysis The process of examining malware without executing it, typically by analyzing its code, structure, and metadata to identify its functionality and potential threats

Malpedia An open, community-driven platform that provides structured information and samples about malware families to support threat intelligence and research. See https://malpedia.caad.fkie.fraunhofer.de/

MalwareBazaar A platform for sharing and downloading malware samples, aimed at aiding cybersecurity researchers and analysts in studying malware. See https://bazaar.abuse.ch/

VirusTotal A free online service that analyses files and URLs for viruses, worms, trojans, and other kinds of malicious content using multiple antivirus engines. See https://www.virustotal.com/

VT See VirusTotal

VX-underground A repository and research collective that archives malware samples, threat intelligence, and cybersecurity resources for educational and research purposes. See https://vx-underground.org/

PE format The file format specification for Portable Executable (PE) files used in Windows operating systems; defines the structure and layout of executable files, object code, and DLLs

PE file A Portable Executable (PE) file, the standard file format for executables, object code, and DLLs in 32-bit and 64-bit versions of Windows operating systems

PE files See PE file

Portable Executable See PE file

PE frame See PEFrame

PEFrame An open-source tool used for static analysis of Portable Executable (PE) files to detect malware and extract useful information

Decompilation The process of translating low-level machine code or bytecode back into a higher-level, human-readable source code

decompile See Decompilation

decompiles See Decompilation

Decompiler See Decompilation

Ghidra An open-source software reverse engineering (SRE) suite developed by the NSA, used for analyzing compiled code, decompilation, and malware analysis

Ghydra See Ghidra

RAG approach See Retrieval-Augmented Generation

RAG method See Retrieval-Augmented Generation

Retrieval-Augmented Generation A technique that combines information retrieval with language generation to improve the accuracy and relevance of AI-generated responses

pg-vector See pgvector

pgvector An extension for PostgreSQL that provides support for vector similarity search, enabling efficient handling of embeddings and machine learning workloads

Postgres See PostgreSQL

PostgreSQL An advanced, open-source relational database management system known for its extensibility and SQL compliance

Python A high-level, interpreted programming language known for its readability, simplicity, and versatility in fields such as web development, data science, automation, and artificial intelligence

Python language See Python

PDM See Python PDM

Python PDM A modern Python package and dependency manager that uses the PEP 582 standard for managing project dependencies and environments. See https://pdm-project.org/

Haystack (Python library) An open-source AI framework designed for building end-to-end pipelines for tasks such as question answering, document retrieval, and natural language processing. See https://haystack.deepset.ai/

Haystack LLM See Haystack (Python library)

Haystack Python library See Haystack (Python library)

Acronyms

AI Artificial Intelligence

API Application Programming Interface

LLM Large Language Model

GPT-4o Generative Pre-trained Transformer 4 Omni
https://openai.com/blog/gpt-4o

DeepSeek-R1 DeepSeek Research Model 1
https://deepseek.com/research

ML Machine Learning

RAG Retrieval-Augmented Generation

Bibliography

[1] Mohd Faizal Ab Razak et al. "The rise of "malware": Bibliometric analysis of malware study". In: *Journal of Network and Computer Applications* 75 (2016), pp. 58–76.

[2] H. S. Anderson and P. Roth. "EMBER: An Open Dataset for Training Static PE Malware Machine Learning Models". In: *ArXiv e-prints* (Apr. 2018). arXiv: 1804.04637 [cs.CR].

[3] Sebastian Berrios et al. "Systematic Review: Malware Detection and Classification in Cybersecurity". In: *Applied Sciences* 15.14 (2025). ISSN: 2076-3417. DOI: 10.3390/app15147747. URL: https://www.mdpi.com/2076-3417/15/14/7747.

[4] Patrice Béchard and Orlando Marquez Ayala. "Reducing Hallucination in Structured Outputs via Retrieval-Augmented Generation". In: *arXiv preprint arXiv:2404.08189*. 2024. URL: https://arxiv.org/abs/2404.08189.

[5] Giancarlo Cassanova, Anderies, and Alexander Agung Santoso Gunawan. "Systematic Literature Review of Artificial Intelligence in Malware Detection". In: *2022 1st International Conference on Software Engineering and Information Technology (ICoSEIT)*. 2022, pp. 18–23. DOI: 10.1109/ICoSEIT55604.2022.10029973.

[6] Andrej Fedák and Jozef Štulrajter. "Fundamentals of static malware analysis: principles, methods and tools". In: *Science & Military Journal* 15.1 (2020), pp. 45–53.

[7] Mohamed Amine Ferrag et al. "Generative AI and Large Language Models for Cyber Security: All Insights You Need". In: (May 2024). DOI: 10.48550/arXiv.2405.12750.

[8] Shota Fujii and Rei Yamagishi. *Feasibility Study for Supporting Static Malware Analysis Using LLM*. 2024. arXiv: 2411.14905 [cs.CR]. URL: https://arxiv.org/abs/2411.14905.

[9] Matthew Gaber, Mohiuddin Ahmed, and Helge Janicke. "Malware Detection with Artificial Intelligence: A Systematic Literature Review". In: *ACM Computing Surveys* 56 (Dec. 2023). DOI: 10.1145/3638552.

[10] Yunfan Gao et al. "Retrieval-augmented generation for large language models: A survey". In: *arXiv preprint arXiv:2312.10997* 2.1 (2023).

[11] Shanshan Han, Salman Avestimehr, and Chaoyang He. "A Guardrail Pipeline for Trustworthy LLM Inferences". In: *arXiv preprint arXiv:2502.08142* (2025). URL: https://arxiv.org/abs/2502.08142.

[12] Zhan Peng Lee, Andre Lin, and Calvin Tan. "Finetune-RAG: Fine-Tuning Language Models to Resist Hallucination in Retrieval-Augmented Generation". In: *arXiv preprint arXiv:2505.10792* (2025). URL: https://arxiv.org/abs/2505.10792.

[13] Patrick Lewis et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: *Advances in Neural Information Processing Systems*. Ed. by H. Larochelle et al. Vol. 33. Curran Associates, Inc., 2020, pp. 9459–9474. URL: https://proceedings.neurips.cc/paper_files/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf.

[14] V. Magesh. "Hallucination-Free? Assessing the Reliability of Leading AI Systems in Legal RAG Tasks". In: *Stanford* (2025). URL: https://arxiv.org/abs/2501.05577.

[15] Sharfah Ratibah Tuan Mat et al. "Towards a systematic description of the field using bibliometric analysis: malware evolution". In: *Scientometrics* 126.3 (2021), pp. 2013–2055. DOI: 10.1007/s11192-020-03834-6. URL: https://doi.org/10.1007/s11192-020-03834-6.

[16] Shivani Metta et al. *Generative AI in Cybersecurity*. 2024. arXiv: 2405.01674 [cs.CR]. URL: https://arxiv.org/abs/2405.01674.

[17] Farzad Nourmohammadzadeh Motlagh et al. *Large Language Models in Cybersecurity: State-of-the-Art*. 2024. arXiv: 2402.00891 [cs.CR]. URL: https://arxiv.org/abs/2402.00891.

[18] Bhaskar V Patil and Rahul J Jadhav. "Computer virus and antivirus software a brief review". In: *International Journal of Advances in Management and Economics* 4.2 (2014), pp. 1–4.

[19] Daniel Plohmann et al. "Malpedia: A Collaborative Effort to Inventorize the Malware Landscape". In: Dec. 2017. DOI: 10.18464/cybin.v3i1.17.

[20] Pranab Sahoo et al. "A systematic survey of prompt engineering in large language models: Techniques and applications". In: *arXiv preprint arXiv:2402.07927* (2024).

[21] Ashish Vaswani et al. *Attention Is All You Need*. 2023. arXiv: 1706.03762 [cs.CL]. URL: https://arxiv.org/abs/1706.03762.

[22] Miuyin Wong et al. "An Inside Look into the Practice of Malware Analysis". In: Nov. 2021, pp. 3053–3069. DOI: 10.1145/3460120.3484759.

[23] Wan Zhang and Jing Zhang. "Hallucination Mitigation for Retrieval-Augmented Large Language Models: A Review". In: *Mathematics* 13.5 (2025), p. 856. DOI: 10.3390/math13050856. URL: https://doi.org/10.3390/math13050856.

[24] Wei Zou et al. *PoisonedRAG: Knowledge Corruption Attacks to Retrieval-Augmented Generation of Large Language Models*. 2024. arXiv: 2402.07867 [cs.CR]. URL: https://arxiv.org/abs/2402.07867.